Dossier d'Architecture Technique (DAT) — FileFlow

Document	Dossier d'Architecture Technique
Produit	FileFlow — Plateforme de gestion de fichiers auto-hébergée
Version	1.1
Date	27 octobre 2026
Statut	Validé
Propriétaire	Driss OUAAZIZ — Software Architect
Classification	Interne

Table des matières

1. Objet et portée
2. Vue d'ensemble
3. Choix technologiques et justification
4. Architecture applicative
5. Architecture des données
6. Architecture de sécurité
7. Architecture d'infrastructure et déploiement
8. Exigences non fonctionnelles
9. Contraintes et hypothèses
10. Références

1. Objet et portée

1.1 Objet

Le présent dossier décrit l'architecture technique de la plateforme **FileFlow**. Il constitue le document de référence pour les décisions structurantes de conception, de sécurité et d'exploitation. Il est destiné aux architectes, aux développeurs, aux ingénieurs d'exploitation et aux responsables sécurité intervenant sur le produit.

1.2 Portée

Le document couvre l'intégralité du périmètre applicatif de FileFlow : le service dorsal (backend), l'application cliente (frontend), le modèle de persistance, les mécanismes de sécurité et la chaîne de déploiement. Il ne traite pas des procédures d'exploitation détaillées, renvoyées au **Dossier d'Exploitation (DEX, document 10)**, ni du contrat d'interface applicative, renvoyé à la **Documentation des API (document 08)**.

1.3 Documents liés

RÉF.	DOCUMENT
05	Registre des décisions d'architecture (ADR)
06	Dossier de sécurité et analyse de risques
08	Documentation des API
10	Dossier d'Exploitation (DEX)
14	Rapport d'audit de sécurité (octobre 2026)
15	Plan directeur des scans et rapports de sécurité

---

2. Vue d'ensemble

FileFlow est une plateforme de gestion de fichiers **auto-hébergée**, conçue pour garantir la **souveraineté des données** : l'ensemble des fonctions s'exécute au sein d'un unique binaire déployé dans l'infrastructure de l'organisation cliente, sans dépendance à un service tiers en ligne.

Le système est distribué sous la forme d'un **mono-binaire** écrit en Rust. Ce binaire assume deux responsabilités simultanées :

- il expose une **API REST** sous le préfixe `/api` ;
- il sert l'**application monopage (SPA)** compilée, avec un mécanisme de repli (*fallback*) strictement confiné au répertoire `dist/`.

Les fonctionnalités principales offertes sont :

- l'authentification par jeton, avec double facteur (TOTP) optionnel ;
- la gestion des utilisateurs et des groupes, assortie d'un contrôle d'accès par rôles et par chemins (RBAC) ;
- les opérations fichiers : navigation, téléversement, téléchargement, renommage, déplacement, suppression avec corbeille, recherche, étiquetage et versionnement ;
- le partage : liens publics (avec mot de passe, expiration et plafond de téléchargements), partage interne entre utilisateurs, dossiers collaboratifs, et liens de dépôt anonymes ;
- la journalisation d'audit et la limitation de débit anti-force brute.

L'ensemble de la logique métier est déterministe.

---

3. Choix technologiques et justification

3.1 Service dorsal

DOMAINE	TECHNOLOGIE	JUSTIFICATION
Langage	Rust (édition 2021)	Sûreté mémoire sans ramasse-miettes, performances natives, absence de classes entières de vulnérabilités mémoire, binaire autonome.
Cadre web	axum 0.7 (Tokio)	Cadre asynchrone performant, typage fort des extracteurs, intégration native avec l'écosystème `tower`.

DOMAINE	TECHNOLOGIE	JUSTIFICATION
Persistance de configuration	SQLite (rusqlite, bibliothèque embarquée)	Base transactionnelle sans serveur, embarquée dans le binaire, mode WAL pour la concurrence lecture/écriture.
Jetons	JSON Web Token (jsonwebtoken 10, backend pur Rust)	Standard éprouvé, session sans état côté serveur, algorithme HS256.
Empreintes de mots de passe	bcrypt	Fonction de dérivation à coût paramétrable, résistante à la force brute.
Double facteur	TOTP (RFC 6238)	Standard interopérable avec les applications d'authentification courantes.
Archives de téléchargement groupé	zip 9, compression *deflate* uniquement	Fonctions par défaut désactivées : les moteurs zstd et bzip2, porteurs d'avis de sécurité récurrents, sont exclus du binaire (et bannis par la politique `cargo-deny`, voir §7.5).

3.2 Application cliente

DOMAINE	TECHNOLOGIE	JUSTIFICATION
Bibliothèque d'interface	React 18 + TypeScript	Écosystème mature, typage statique, composants réutilisables.
Outil de construction	Vite	Compilation rapide, découpage de code, sortie statique optimisée.
Gestion d'état	Zustand	Gestion d'état légère et sans boilerplate.
Style	Tailwind CSS	Cohérence visuelle, absence de feuilles de style dispersées.
Assainissement HTML	DOMPurify	Défense en profondeur contre l'injection de script lors du rendu de contenu enrichi.

3.3 Traitement de fichiers bureautiques

Le rendu des documents Word (`.docx`) et des classeurs (`.xlsx`) est réalisé **côté client**, sans dépendance de rendu lourde côté serveur. Les présentations (`.pptx`) ne bénéficient pas d'aperçu intégré et sont proposées au téléchargement, décision documentée dans le registre des décisions d'architecture (document 05).

---

4. Architecture applicative

4.1 Organisation en couches

L'application s'organise selon une séparation nette des responsabilités :

1. **Couche de présentation** — le SPA React, servi statiquement, consomme l'API REST.
2. **Couche d'exposition HTTP** — le routeur axum, les intergiciels (*middlewares*) transversaux et les gestionnaires de requêtes.
3. **Couche de domaine** — les magasins (*stores*) encapsulant les règles métier et l'état applicatif.
4. **Couche de persistance** — l'accès à SQLite et au système de fichiers.

4.2 Point d'entrée et état applicatif

Le binaire est structuré en une bibliothèque (`lib`) contenant l'intégralité de la logique, et un exécutable (`bin`) réduit à un lanceur. Cette séparation permet aux campagnes de test et de *fuzzing* de lier le code en tant que bibliothèque.

- **`Config`** — configuration immuable dérivée de l'environnement (répertoires, secret de signature, tailles maximales de téléversement authentifié et de dépôt public, options de proxy et de cookies).
- **`AppState`** — état partagé, cloné à moindre coût, agrégeant les magasins, le journal d'audit, les limiteurs de débit, les verrous de cohérence et le sémaphore de téléchargement groupé.
- **`run`** — fonction d'amorçage : initialisation de la journalisation, ouverture de la base, chargement des magasins, création de l'administrateur initial, construction du routeur, lancement du serveur et gestion de l'arrêt gracieux.

4.3 Intergiciels transversaux

INTERGICIEL	RÔLE
`require_auth`	Validation du jeton, vérification de l'existence et du statut de l'utilisateur, contrôle de la version de jeton, injection des permissions résolues.
`require_admin`	Restriction aux administrateurs, appliqué après `require_auth` .
En-têtes de sécurité	Application systématique des en-têtes de sécurité HTTP (voir §6).
Compression	Compression des réponses.
Journalisation des traces	Traçabilité des requêtes HTTP.

4.4 Modules du domaine

MODULE	RESPONSABILITÉ
`auth`	Création et vérification des jetons de session et des jetons d'attente 2FA ; intergiciels d'autorisation.
`two_factor`	Parcours TOTP : activation, confirmation, vérification à la connexion, codes de récupération, désactivation avec double vérification (*step-up*) et rotation de session.
`users`	`UserStore` : cycle de vie des utilisateurs et des groupes, RBAC, gestion TOTP, versionnement de jeton pour la révocation.
`files`	Normalisation canonique des chemins virtuels (`normalize_virtual` , `path_covers`) et points de contrôle d'accès (`validate_access` , `validate_own_access` , `own_grant`), résolution anti-traversée des chemins, téléversement multipart, téléchargement avec plages d'octets (RFC 7233), prévisualisation avec neutralisation du contenu actif, création de répertoire, renommage, déplacement, suppression vers corbeille, recherche, étiquetage et versionnement.
`uploads`	Téléversement fragmenté et reprenable, avec sessions en mémoire et récupérateur de sessions périmées.
`shares`	Liens de partage publics : mot de passe optionnel, expiration, plafond de téléchargements réservé de façon atomique, revérification des droits du créateur à chaque téléchargement.
`collab`	Dossiers collaboratifs et gestion des membres, avec autorité dérivée des droits propres du créateur.
`internal_shares`	Partage direct de ressources entre utilisateurs authentifiés.

MODULE	RESPONSABILITÉ
`favorites`	Marque-pages personnels de fichiers et de dossiers.
`upload_requests`	Liens de dépôt permettant à des tiers non authentifiés de déposer des fichiers, sans jamais écraser un fichier existant.
`audit`	Journal d'audit append-only au format JSONL, avec rotation.
`ratelimit`	Verrous anti-force brute en mémoire : connexion (par IP, par couple utilisateur/IP, par compte), second facteur, mots de passe des liens publics.
`db`	Abstraction du magasin documentaire SQLite.
`totp`	Génération et vérification des codes TOTP.
`models`	Types de données partagés.
`error`	Type d'erreur unifié et conversion en réponses HTTP.

4.5 Frontend

L'application cliente est structurée autour d'un wrapper d'appel réseau (`api.ts` , transmission systématique des cookies de session), d'un magasin d'état (`store.ts`), et de composants de visualisation (`FileViewer` , `PreviewPanel`). Le rendu de contenu enrichi (Markdown, coloration syntaxique, documents bureautiques) passe par des modules dédiés (`markdown.ts` , `syntax.ts` , `office.ts`) dont la sortie est systématiquement assainie par `sanitize.ts` .

---

5. Architecture des données

5.1 Modèle de persistance : magasin documentaire

FileFlow persiste son état de configuration dans une base **SQLite** unique (`.fileflow.db` , mode WAL). Le modèle retenu est celui d'un **magasin documentaire** : chaque famille d'entités dispose d'une table dont chaque ligne associe une **clé primaire** à un **document JSON**.

Le fonctionnement repose sur un **cache mémoire en écriture immédiate** (*write-through*) :

- à la lecture, les entités sont servies depuis une table de hachage en mémoire (`HashMap`) ;
- à chaque mutation, une **unique ligne** est réécrite dans SQLite, supprimant les réécritures intégrales de fichiers pratiquées auparavant.

La fonction d'hydratation (`hydrate_map`) charge chaque magasin au démarrage et, à première exécution, importe automatiquement les anciens fichiers JSON s'ils existent, assurant une migration transparente.

5.2 Entités de configuration

ENTITÉ	CLÉ	DESCRIPTION
`StoredUser`	nom d'utilisateur	Compte : empreinte du mot de passe, rôle, statut, permissions directes, groupes, quota, octets consommés, métadonnées, secret et codes de récupération TOTP, version de jeton.
`UserGroup`	identifiant	Groupe : nom, permissions, chemins autorisés.
`ShareLink`	jeton	Lien public : chemin cible, empreinte de mot de passe optionnelle, expiration, plafond et compteur de téléchargements.
`SharedFolder`	identifiant	Dossier collaboratif : chemin, propriétaire, liste des membres et de leurs droits.

ENTITÉ	CLÉ	DESCRIPTION
`InternalShare`	identifiant	Partage direct : chemin, propriétaire, destinataire, droit d'écriture.
`Favorite`	utilisateur + chemin	Marque-page personnel.
`UploadRequest`	jeton	Lien de dépôt anonyme : dossier cible, expiration, mot de passe optionnel.

Les chemins persistés (liens publics, demandes de dépôt, partages internes, dossiers collaboratifs, favoris, chemins autorisés des groupes) sont enregistrés sous leur **forme canonique** (voir §6.2.1) : les comparaisons de préfixe et le suivi des renommages opèrent ainsi sur la chaîne exacte qui sera accédée. Au chargement, les partages internes et dossiers collaboratifs antérieurs sont canonisés, et ceux qui ne peuvent l'être sont écartés ; une entrée non canonique des chemins autorisés d'un groupe n'accorde aucun accès.

5.3 Données propres au système de fichiers

Les données liées au système de fichiers ne résident pas en base mais dans le système de fichiers lui-même, sous le répertoire interne `.fileflow/` :

DONNÉE	EMPLACEMENT	NATURE
`FileMetadata`	`.fileflow/metadata.json`	Étiquettes et collections associées aux fichiers, protégé par un verrou de cohérence.
`TrashEntry`	`.fileflow/trash.json` + `.fileflow/trash/`	Éléments supprimés (chemin d'origine, date, auteur, taille), avec purge automatique après 30 jours.
`VersionEntry`	`.fileflow/versions/`	Versions antérieures archivées lors des écrasements.
Sessions de téléversement	`.fileflow/uploads/`	Fichiers partiels `.part` des téléversements fragmentés.
`AuditEntry`	`.fileflow_audit.json`	Journal d'audit append-only (JSONL).

5.4 Séparation des espaces de stockage

Deux espaces sont distingués :

- `ROOT_DIR` — l'espace de données des utilisateurs, seul exposé aux opérations fichiers ;
- `CONFIG_DIR` — l'espace de configuration (base SQLite, secret de signature), par défaut confondu avec `ROOT_DIR` mais dissociable.

Le répertoire interne `.fileflow/` et les fichiers `.fileflow*` sont **inaccessibles** aux chemins utilisateurs à quelque profondeur que ce soit (voir §6.4).

---

6. Architecture de sécurité

Cette section synthétise les mécanismes de sécurité. L'analyse de risques complète figure dans le **Dossier de sécurité (document 06)**.

6.1 Authentification

L'authentification repose sur un **JWT** (algorithme HS256) transporté dans un cookie `HttpOnly`, `SameSite=Strict` et `Secure` (paramétrable). L'en-tête `Authorization: Bearer` est également accepté. Le

secret de signature est fourni par l'environnement ou généré au premier démarrage (256 bits), puis persisté avec des droits restreints au propriétaire.

Le double facteur TOTP est optionnel : lorsqu'il est activé, l'authentification par mot de passe délivre un **jeton d'attente** de courte durée, échangé contre une session complète après vérification du code. L'authentification par mot de passe est **égalisée en temps** (vérification bcrypt factice pour un utilisateur inconnu) afin de ne pas révéler l'existence d'un compte.

Toute modification d'un second facteur déjà actif exige une **double vérification** (*step-up*) : la désactivation du TOTP et la régénération des codes de récupération requièrent le mot de passe **et** un code valide (TOTP courant ou code de récupération, alors consommé) ; une session dérobée seule ou un mot de passe dérobé seul ne suffisent pas. Les échecs alimentent le verrou du second facteur, et la réponse d'erreur est identique que le mot de passe ou le code soit en cause. L'initialisation et la confirmation du TOTP sont refusées lorsque le double facteur est déjà actif.

La déconnexion (`POST /api/auth/logout`) est une route publique : elle aboutit même avec un jeton absent, expiré ou révoqué, et efface systématiquement les deux cookies (`fileflow_token` et `fileflow_2fa_pending`).

6.2 Contrôle d'accès (RBAC)

Le contrôle d'accès s'articule autour de rôles (Administrateur, Manager, Éditeur, Lecteur, Utilisateur) et de permissions (`FileRead` , `FileWrite` , `FileDelete` , `UserManage` , `GroupManage` , `ViewAudit`). Les permissions effectives d'un utilisateur résultent de l'union de ses permissions directes et de celles de ses groupes, avec repli sur les permissions par défaut du rôle.

La fonction `validate_access` constitue le **point de contrôle unique** des autorisations d'accès aux ressources : administrateur, puis partage interne, puis permissions résolues, puis appartenance à un dossier collaboratif, puis chemins autorisés par groupe.

6.2.1 Chemins canoniques

Toutes les règles d'autorisation comparent des chemins virtuels sous forme de chaînes ; la chaîne contrôlée doit donc être exactement celle qui sera accédée. Le chemin fourni par le client est d'abord ramené à sa **forme canonique** par `files::normalize_virtual` (`/a/b` , segments vides et `.` supprimés, tout segment `..` rejeté), puis la couverture d'un chemin par un préfixe est évaluée **segment par segment** par `files::path_covers` : `/equipeA` couvre `/equipeA/x` mais pas `/equipeA-archives` . Aucune comparaison brute par préfixe de chaîne n'est employée. Les chemins autorisés des groupes sont validés et canonisés à l'écriture (`canonical_prefix` en lecture, qui fait échouer en mode fermé toute valeur malformée).

Un partage, un lien ou un favori ne peut être créé sur une ressource atteinte au travers d'un lien symbolique (`ensure_direct_path`) : le chemin autorisé et le chemin réel doivent coïncider pour que la purge et le rebasage ultérieurs s'appliquent correctement.

6.2.2 Accès propre et autorité dérivée

Trois points de contrôle distincts sont exposés :

FONCTION	SOURCES D'ACCÈS PRISES EN COMPTE	USAGE
`validate_access`	Rôle, permissions, groupes, dossiers collaboratifs, partages internes	Accès ordinaire à une ressource.
`validate_own_access`	Idem, **sans** les partages internes	Octroi d'un nouveau partage interne ; création d'une demande de dépôt (`FileWrite` exigé).

FONCTION	SOURCES D'ACCÈS PRISES EN COMPTE	USAGE
`own_grant`	Rôle, permissions et groupes **uniquement** (ni partages internes, ni dossiers collaboratifs)	Borne de ce qu'un utilisateur peut transmettre à autrui.

Les **dossiers collaboratifs** reposent sur une autorité dérivée : un membre n'obtient jamais davantage que les droits propres (`own_grant`) du créateur du dossier, vérifiés **à chaque accès** et non à la seule création. Révoquer les droits du créateur révoque donc ceux des membres. Créer un dossier collaboratif exige `own_grant(FileRead)` sur le chemin ; accorder l'écriture à un membre exige `own_grant(FileWrite)` . Un accès obtenu par partage ne peut ainsi jamais être re-partagé de façon transitive.

6.2.3 Protection du dernier administrateur

La rétrogradation, la désactivation ou la suppression du **dernier administrateur actif** est refusée (réponse 400) ; un administrateur désactivé n'est pas compté. Le contrôle s'exécute sous le même verrou que la modification (`UserStore::apply_update` , `UserStore::delete`), ce qui exclut toute situation de concurrence entre deux requêtes simultanées.

6.3 Révocation de session

Chaque utilisateur porte une **version de jeton** (`token_version`) incorporée au JWT. Toute incrémentation invalide instantanément l'ensemble des jetons émis. Les déclencheurs sont : le changement de mot de passe, la déconnexion de tous les appareils, la révocation administrative, toute modification d'autorisation (rôle, permissions, groupes, statut), ainsi que la confirmation et la désactivation du double facteur. Dans ces deux derniers cas, la session courante est **renouvelée** (nouveau cookie émis avec la nouvelle version) tandis que toutes les autres sessions sont révoquées.

La version de jeton initiale d'un compte est **tirée aléatoirement** (entier de 64 bits) : un jeton émis pour un compte supprimé ne redevient pas valide pour un compte recréé ultérieurement sous le même nom.

6.4 Confinement des chemins

La fonction `resolve` normalise les chemins fournis par l'utilisateur **sans accès au système de fichiers** pour la partie normalisation, rejette toute tentative de traversée (`..`), interdit l'accès aux entrées internes `.fileflow*` à toute profondeur, et vérifie le confinement dans la racine canonique. Lorsque la cible existe, elle est canonisée et sa containment est re-vérifiée, neutralisant les échappements par lien symbolique.

6.5 En-têtes de sécurité et surface réseau

Toutes les réponses portent une politique de sécurité de contenu (CSP) stricte, l'interdiction du reniflage de type (`nosniff`), la protection contre le détournement de cadre (`X-Frame-Options`), une politique de référent restrictive et, en contexte TLS, l'en-tête HSTS. Aucune politique CORS permissive n'est configurée. Le contenu actif téléversé (HTML, SVG, XML) est neutralisé en `text/plain` lors de la prévisualisation.

6.6 Liens publics et demandes de dépôt

Les liens publics (`/s/:token`) et les demandes de dépôt (`/r/:token`) sont servis sans authentification ; leur sécurité repose sur les règles suivantes :

- **Revérification du créateur.** À chaque téléchargement par lien public, le serveur vérifie que le créateur est toujours actif et dispose toujours de l'accès en lecture au fichier ; à chaque dépôt, que le créateur de la demande détient toujours l'accès en écriture propre (`validate_own_access(FileWrite)`) au dossier cible. Tout échec produit une réponse 404 uniforme qui ne révèle pas la cause.

- **Création.** Une demande de dépôt ne peut être créée que sur un dossier où l'utilisateur dispose de l'écriture **en propre**, et non au travers d'un partage. La création d'un lien public exige l'accès en lecture via `validate_access`, partages internes compris ; ce point est suivi comme constat ouvert dans le plan des scans et rapports de sécurité (document 15).
- **Cycle de vie.** Lors d'un renommage ou d'un déplacement, les liens publics et demandes de dépôt suivent la ressource (rebasage du chemin, comme les partages internes, dossiers collaboratifs, favoris et métadonnées) ; lors d'une suppression, ils sont supprimés. Les liens publics expirés sont en outre purgés périodiquement.
- **Plafond de téléchargements.** La vérification et l'incrémentation du compteur s'effectuent de façon **atomique** sous un même verrou (`ShareStore::try_reserve_download`) ; la réservation est restituée si l'envoi échoue. Un lien plafonné ignore l'en-tête `Range`, faute de quoi un lien à usage unique pourrait être téléchargé indéfiniment par morceaux ; un lien sans plafond continue de servir les requêtes partielles (206).
- **Dépôts anonymes.** Pour une demande protégée, le champ mot de passe doit précéder les fichiers dans le corps multipart : un fichier reçu avant la vérification du mot de passe est refusé, de sorte qu'un mot de passe erroné ne provoque aucune écriture disque. Les fichiers sont reçus en flux dans des fichiers temporaires, nettoyés en cas d'erreur, puis placés **sans jamais écraser** un fichier existant : un nom déjà pris est numéroté (`rapport (1).txt`), réservé de façon atomique (création exclusive) puis renommé. La taille d'un dépôt est bornée par `PUBLIC_UPLOAD_MAX_MB` (voir §7.4).

6.7 Limitation des tentatives

Les verrous anti-force brute sont tenus **en mémoire** (perdus au redémarrage) et purgés périodiquement des entrées devenues sans effet. Chaque verrou s'arme après un nombre d'échecs donné et dure 15 minutes (`LOCKOUT_SECS`) ; une tentative refusée par un verrou reçoit une réponse 429.

LIMITEUR	CLÉ	SEUIL	REMISE À ZÉRO
`ip_limiter`	Adresse IP	5 échecs (`MAX_ATTEMPTS`)	Connexion réussie depuis cette IP.
`user_ip_limiter`	Couple (utilisateur, IP)	5 échecs	Connexion réussie pour ce couple.
`user_limiter`	Compte, toutes IP confondues	50 échecs (`ACCOUNT_MAX_ATTEMPTS`)	Aucune : le compteur expire seul 15 minutes après le dernier échec, une connexion réussie intercalée ne le réinitialise pas.
`totp_limiter`	Compte	5 échecs (second facteur et vérifications *step-up*)	Code valide.
`link_limiter`	`link:<jeton>` et `ip:<adresse>`	5 échecs sur l'une ou l'autre clé	Mot de passe correct : seule la clé du lien est effacée.

Ce modèle empêche un tiers de bloquer un compte au moyen de cinq mots de passe erronés : le verrou par couple n'affecte que le client fautif, et le verrou de compte exige une cinquantaine d'échecs répartis sur au moins dix adresses. Les échecs sur un compte inconnu sont comptabilisés à l'identique, afin que le verrouillage ne permette pas d'énumérer les comptes. Pour les liens publics, la clé par lien borne une recherche répartie sur de nombreuses adresses et la clé par adresse borne un client qui balaierait de nombreux liens ; la clé par lien permet en revanche à un tiers de verrouiller temporairement un lien pour tous ses destinataires, constat ouvert suivi dans le document 15.

---

7. Architecture d'infrastructure et déploiement

7.1 Construction en plusieurs étapes

L'image est construite selon un `Dockerfile` multi-étapes :

1. **Construction du frontend** — compilation du SPA React en artefacts statiques.
2. **Construction du backend** — compilation du binaire Rust pour la cible **`x86_64-unknown-linux-musl`**, en liaison **entièrement statique** (`crt-static` et modèle de relocation statique), afin de produire un exécutable sans dépendance à une bibliothèque C dynamique.
3. **Image d'exécution** — assemblage du binaire et des artefacts statiques sur une base minimale.

7.2 Image d'exécution

L'image finale repose sur **`distroless/static`** en utilisateur **`nonroot`**. Cette base ne contient ni interpréteur de commandes, ni gestionnaire de paquets, ni bibliothèque système : la surface d'attaque et le nombre de vulnérabilités hérités de la base sont ainsi réduits au strict minimum. La combinaison binaire statique musl + base statique élimine l'exposition aux vulnérabilités des bibliothèques glibc et OpenSSL.

7.3 Orchestration et exploitation

Le déploiement de référence s'appuie sur **`docker-compose`**, avec :

- des **volumes** distincts pour les données (`ROOT_DIR`) et la configuration (`CONFIG_DIR`), garantissant la persistance ;
- une **sonde de santé** auto-portée : la commande `fileflow healthcheck` interroge le point `/api/health` sans dépendre d'outils externes absents de l'image distroless.

7.4 Configuration par environnement

VARIABLE	RÔLE
`ROOT_DIR`	Répertoire des données utilisateurs.
`CONFIG_DIR`	Répertoire de configuration (base, secret).
`ADMIN_USERNAME` / **`ADMIN_PASSWORD`**	Identifiants de l'administrateur initial.
`JWT_SECRET`	Secret de signature des jetons (généré si absent).
`MAX_UPLOAD_MB`	Taille maximale de téléversement, en Mo (200 par défaut).
`PUBLIC_UPLOAD_MAX_MB`	Taille maximale d'un dépôt anonyme via une demande de dépôt, en Mo (100 par défaut, plafonnée à `MAX_UPLOAD_MB`).
`PORT`	Port d'écoute (8080 par défaut).
`COOKIE_SECURE`	Attribut `Secure` des cookies (activé par défaut).
`TRUST_PROXY_HEADERS`	Prise en compte des en-têtes `X-Forwarded-For` / `X-Real-IP` pour l'adresse cliente (désactivée par défaut) ; à n'activer que derrière un proxy qui écrase ces en-têtes, l'adresse cliente alimentant le journal d'audit et les verrous anti-force brute.

Les procédures d'exploitation détaillées (sauvegarde, restauration, montée de version) sont décrites dans le **Dossier d'Exploitation (document 10)**.

7.5 Intégration continue et outillage qualité

Deux chaînes d'intégration continue s'exécutent sur chaque poussée et demande de fusion :

- `ci.yml` **(correction)** — côté frontend : `pnpm lint` (ESLint avec `typescript-eslint`, `@eslint-react`, `eslint-plugin-security` et `eslint-plugin-no-unsanitized` ; les puits XSS et les expressions régulières vulnérables au déni de service sont des erreurs bloquantes), contrôle des traductions, tests et construction ; côté backend : `cargo clippy --all-targets --locked -- -D warnings` puis `cargo test --locked`. Les lints `unwrap_used`, `dbg_macro` et `todo`, déclarés dans `Cargo.toml`, deviennent ainsi bloquants ; `unwrap` reste admis dans les tests (`clippy.toml`).
- `security.yml` **(sécurité)** — également planifiée chaque lundi à 06:00 UTC : semgrep, cargo-audit, **cargo-deny** (licences permissives en liste blanche, sources limitées à crates.io, crates zstd et bzip2 bannies), pnpm audit, trivy (système de fichiers et image), gitleaks, et *fuzzing* `cargo-fuzz` des quatre cibles d'analyseurs.

Les avis de sécurité non atteignables sont neutralisés explicitement et justifiés dans `backend/.cargo/audit.toml` et `backend/deny.toml` : RUSTSEC-2023-0071 (`rsa`) et RUSTSEC-2026-0387 (`crypto-bigint`), tous deux tirés par le backend cryptographique de `jsonwebtoken` alors que seuls des jetons HS256 sont émis et vérifiés.

Les dépendances sont suivies par **Dependabot** (`.github/dependabot.yml`) pour les écosystèmes cargo, npm, GitHub Actions et Docker : mises à jour hebdomadaires (le lundi), groupées pour les versions mineures et correctives, avec un délai de carence de 7 jours avant de proposer une version publiée ; les correctifs de sécurité sont proposés dès la publication de l'avis.

---

8. Exigences non fonctionnelles

DOMAINE	EXIGENCE	CIBLE
Performance	Latence des opérations de lecture servies depuis le cache mémoire.	Réponse inférieure à 100 ms hors transfert de données.
Performance	Téléchargement et téléversement de gros fichiers sans saturation mémoire.	Diffusion en flux, empreinte mémoire constante indépendante de la taille du fichier.
Sécurité	Absence de vulnérabilité connue non traitée dans les dépendances et l'image.	Scans (SAST, dépendances, image, secrets) sans constat non traité.
Sécurité	Résistance à la force brute sur l'authentification et les liens publics.	Verrou de 15 minutes après 5 échecs par IP, par couple utilisateur/IP, sur le second facteur et par lien public ; après 50 échecs par compte toutes IP confondues (voir §6.7).
Disponibilité	Arrêt sans interruption des requêtes en cours.	Arrêt gracieux sur signal de terminaison.
Disponibilité	Détection d'état par l'orchestrateur.	Sonde de santé non authentifiée sur `/api/health`.
Portabilité	Exécution sans dépendance système.	Binaire statique, image distroless.
Maintenabilité	Séparation bibliothèque/exécutable, tests unitaires colocalisés.	Couverture des fonctions critiques par tests et *fuzzing*.
Exploitabilité	Persistance durable et sauvegardable.	Données et configuration sur volumes dédiés.
Exploitabilité	Croissance maîtrisée des journaux.	Rotation automatique du journal d'audit.

---

9. Contraintes et hypothèses

9.1 Contraintes

- Le produit est **auto-hébergé** ; l'organisation cliente assume l'exploitation et la responsabilité de traitement des données.
- La terminaison **TLS** est assurée par un reverse-proxy en amont ; le service n'assure pas lui-même le chiffrement de transport.
- La base SQLite implique un **unique processus écrivain** ; le modèle vise un déploiement mono-instance. Une montée en charge en écriture concurrente élevée nécessiterait une évolution de la couche de persistance.

9.2 Hypothèses

- Le service est déployé derrière un reverse-proxy de confiance lorsque `TRUST_PROXY_HEADERS` est activé ; à défaut, les en-têtes de proxy sont ignorés pour prévenir l'usurpation d'adresse.
- Les volumes de données et de configuration sont sauvegardés selon la politique de l'organisation.
- Les fichiers déposés peuvent contenir des données à caractère personnel ; la conformité correspondante est traitée dans le dossier de conformité RGPD.

---

10. Références

- **ISO/IEC/IEEE 42010** — *Systems and software engineering — Architecture description.*
- **arc42** — Gabarit de documentation d'architecture (arc42.org).
- **TOGAF ADM** — *The Open Group Architecture Framework — Architecture Development Method* (The Open Group).
- **RFC 7233** — *HTTP/1.1 Range Requests.*
- **RFC 6238** — *TOTP: Time-Based One-Time Password Algorithm.*